

Kai Yi Ng

Assignment 6: Fuzz Tester

Repo: <https://github.com/kaiyi03/CWM-Project/tree/main/assignment6>

AI Declaration:

Model used: Claude Sonnet 4.6

1. Generate and debug code.
2. Help tweak the phrasing of written paragraphs to make it more understandable.
3. Help discuss the big picture workflow for my project.

<https://claude.ai/share/bf465f37-6316-4640-b968-e57f9341e10a>

Introduction

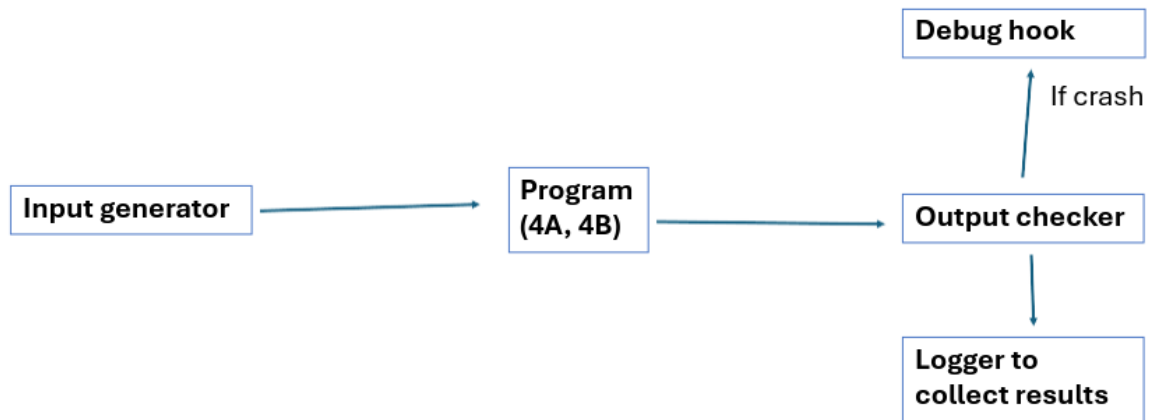
In Exercise 4, I manually identified a buffer overflow vulnerability in a small C program by analysing its stack layout, calculating a precise offset (24 bytes) needed to overwrite a return address, and using this to redirect control flow from `victim()` to `steal_password()`. I also identified, through manual testing, that a second program (`vuln`) crashes when supplied with a sufficiently long input at one of its prompts.

This project automates that process of vulnerability discovery using fuzz testing — a technique that repeatedly supplies a program with invalid, unexpected, or random data and monitors for crashes or other unexpected behaviour.

The aim is to investigate two questions. First, whether an automated fuzzer can rediscover the vulnerabilities identified manually in Exercise 4 without the fuzzer being given prior knowledge of their location. Second, by how much incorporating prior knowledge of the program (such as the offset-24 finding) into the fuzzer's input generation strategy improves its efficiency compared to purely random testing.

The fuzzer targets the two vulnerable programs from Exercise 4 (4A/main and 4B/vuln) and uses two input generation strategies: random, which samples inputs uniformly across the valid range, and boundary-focused, which concentrates testing near values identified in Exercise 4 as marking a transition in program behaviour (e.g., offset 24 for 4A, and input lengths around the segfault threshold for 4B). The fuzzer also incorporates a debugging hook that allows the root cause to be identified programmatically rather than by hand. Results are logged and analysed to compare the efficiency of the two strategies — specifically, the number of trials each requires to find a crash.

Design



Files in the Repo

Target_4A.py - It's a Python function that automates running the offset-sweep we did manually in Exercise 4. Instead of me typing ./main 24 and reading the output myself, the function does it and classifies the result automatically.

Target_4B.py - It automates the manual interaction from Exercise 4. Instead of pasting a long string into the password/command prompt by hand and watching for a segfault, the function feeds the whole conversation to the program programmatically and checks whether it crashed.

Generators.py - contains 4 functions. 2 for 4A and 4B each respectively. The Boundary functions encodes the results we found manually in exercise 4. (24 for 4A and 264 for 4B). The code uses a Gaussian distribution to get a spread around the exact known value.

Logger.py - writes results to .csv files

Debug_hook.py - this module automates the debugging and points out where the crash occurred.

Runner.py - this is the script that is actually ran from the terminal. “campaign” refers to a combination of a target (4A or 4B) and a strategy (random or boundary).

Analysis.py - turns the raw csv files into numbers

Method and Results

For each combination of target + strategy, I will be running 3 runs. Each run contains 150 trials.

We have a total of 4 combinations:

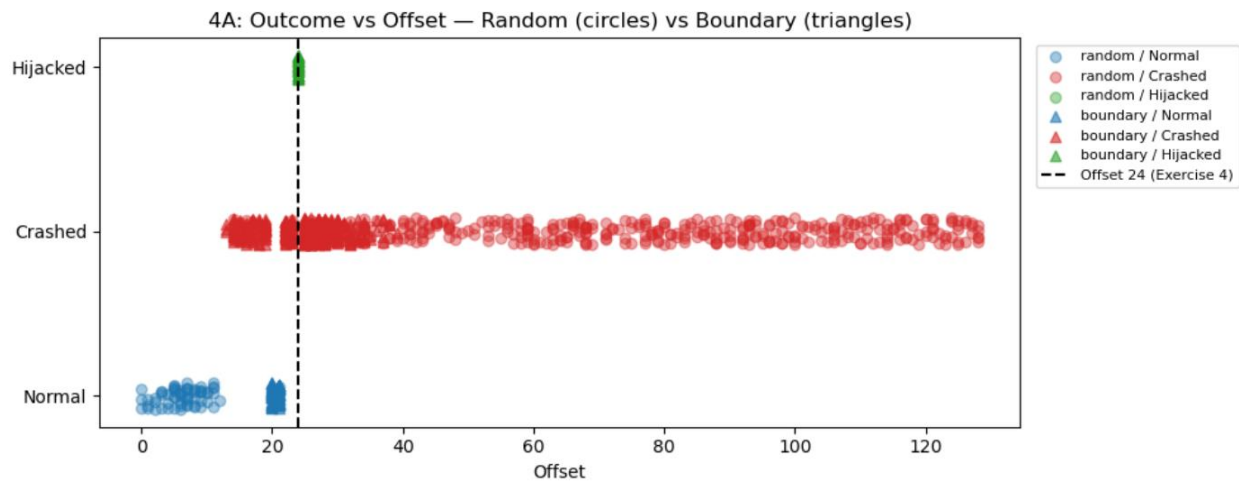
1. Target4A + random
2. Target4A + boundary
3. Target4B + random
4. Target4B + boundary

I will be analyzing three graphs:

1. For 4A, Outcome vs Offset. [Crash = 1, normal = 0]
2. For 4B, Outcome vs Payload Length
3. Crash/Hijack Rate Comparison for Each Strategy to Compare the Improvement in Efficiency for Random vs Boundary Input Generation.

Results and Discussion

4A: Outcome vs Offset



The Boundary Strategy for 4A is centred around offset 24 which can be pre-calculated using stack arithmetic.

Green symbols (hijacked) match expectations from exercise 24. The `steal_password()` is successfully called when the offset is 24. This is the same for both strategies (as expected since the offset value does not change depending on the strategy used) hence the green circles for both strategies perfectly overlap.

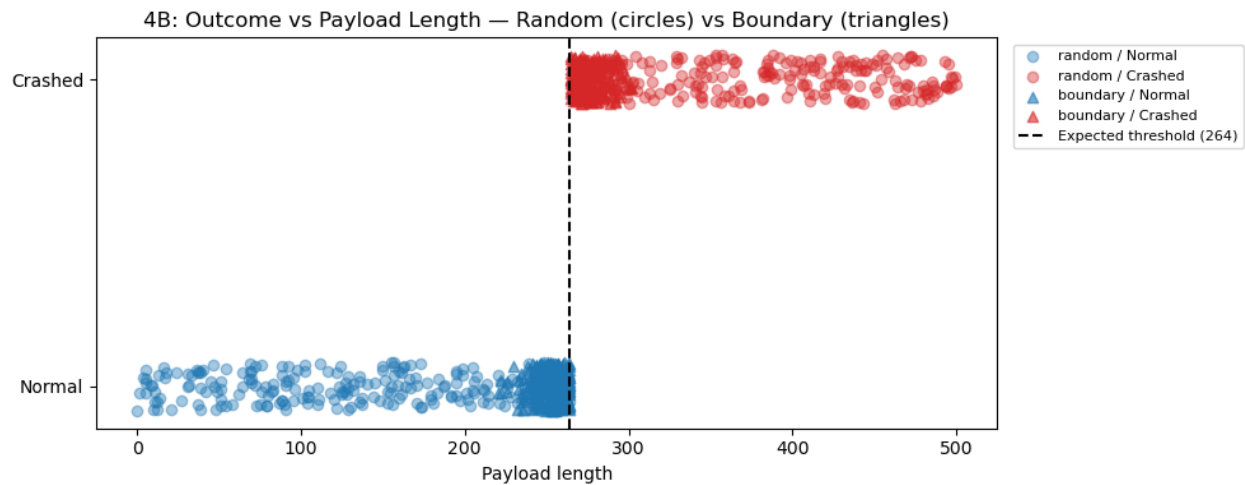
I also note that the crash outcomes happen slightly before offset 24. This reflects corruption that causes a crash without achieving the desired hijack. So, there is a difference between crash detection and exploit detection. Crash means something went wrong but not necessarily that the attacker gained control.

From exercise 4 (README file):

```
Higher addresses (toward top of address space)
+-----+
| return address | <-- where to go when the function returns
+-----+
| saved RBP      | <-- caller's base pointer
+-----+
| local variables |
+-----+
| buf[16]        | <-- arrays live here
+-----+
Writing to buf moves stack pointer UPWARDS
...
```

As the offset increases from 0, the written bytes land progressively further into the frame. This explains the “partial corruption” we observe because at offset slightly less than 24, the saved registers get corrupted first.

4B: Outcome vs Payload Length



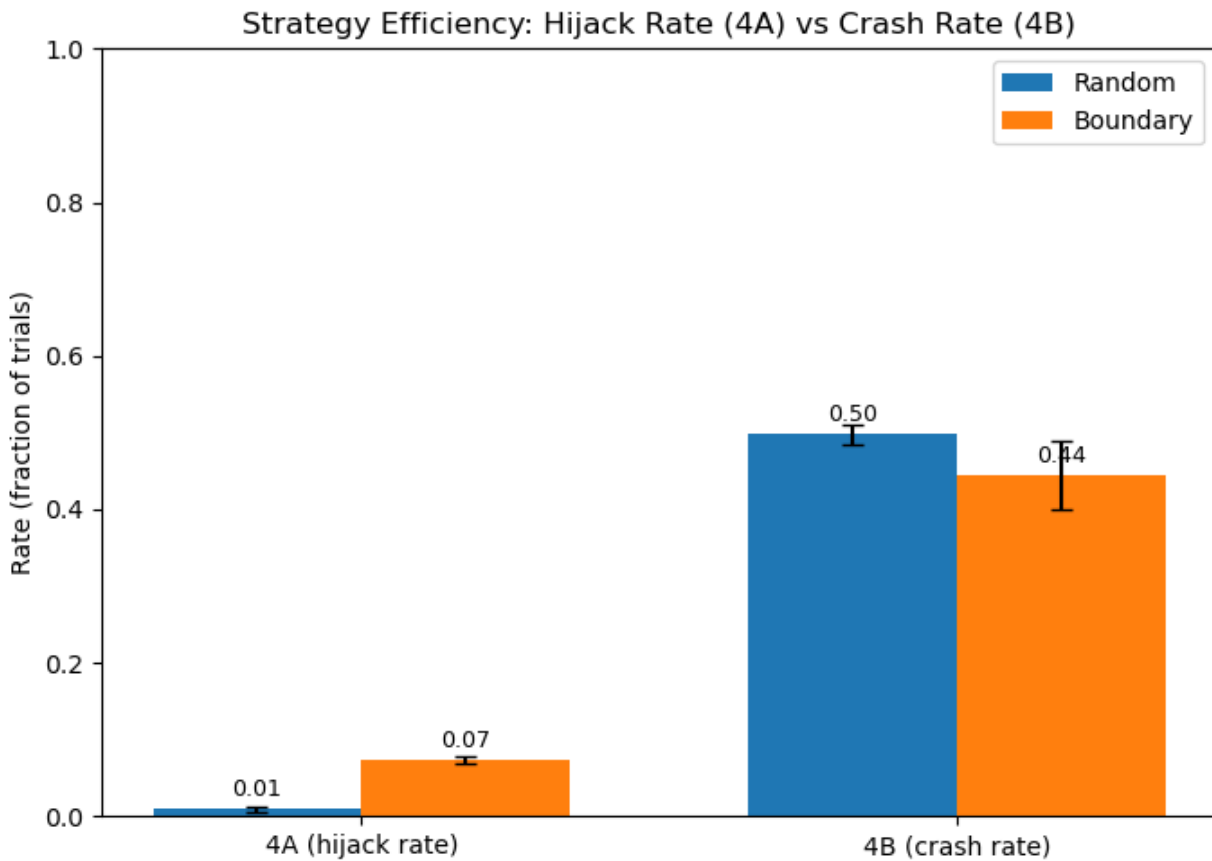
There is a step change from "Normal" to "Crashed" at payload length approx. 264, confirming the experimentally derived threshold from Exercise 4.

No crashes occur below 264, and all trials above 264 crashes. Unlike 4A, where crashes start below the critical value, there is a sharp transition between "Normal" and "Crashed". This reflects a different stack frame in vuln.c. Once the payload exceeds the buffer, the return address is corrupted immediately, with no partial corruption.

In the Boundary Strategy for 4B, I centered the distribution at 264 because of the experimental data that we got from exercise 4. In the real world, if we do not have this experimental data, we would probably first run some trials using the random strategy and based on the scatter plot, identify roughly where the crash occurred. We then set our boundary centered about that point.

So, this boundary strategy is more of a refinement tool that finds it more efficiently if you already know where to look.

4C: Strategy Efficiency Comparison



4A results for random strategy is an extremely small value close to zero. Which is expected since the correct offset is only 24. And there is a total of 129 offsets.

So, the total probability of hijacking per trial (assuming random) is only going to be $1/129$.

Boundary strategy performs a 7x improvement for 4A. Prior knowledge of exploit location significantly improves the fuzzing efficiency when the event is rare.

For 4B, the crash rate is similar for both strategies. The random strategy has crash rate of 0.5 which is expected. Because the random strategy generates random input lengths from 0 – 500. Since the crash threshold is 264, the probability of crash is $(500 - 264)/500 \sim 47\%$.

Boundary crashing rate is also expected to be 50% since it is centred on the sampling threshold.

So for 4B, boundary strategy doesn't seem to provide significant improvement compared to random. The result suggest that boundary focused fuzzing provides the greatest efficiency gain when the target event occupies a small fraction of the total possible input space that the fuzzer is allowed to generate. (e.g., in 4A)

4A: only 1 out of 129 inputs triggers the hijack (0.8%). Random sampling has a very low probability of hitting it, so boundary strategy makes a big difference

4B: 237 out of 501 inputs trigger a crash (47%). Random sampling hits the crash roughly half the time anyway, so there's very little room for boundary to improve on that

Debug Hook

```
This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) [answered N; input not from terminal]
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGILL, Illegal instruction.
0x00007ffffffffffde82 in ?? ()
#0 0x00007ffffffffffde82 in ?? ()
#1 0x00000000200400000 in ?? ()
#2 0x4141414141414141 in ?? ()
#3 0x4141414141414141 in ?? ()
#4 0x4141414141414141 in ?? ()
#5 0x000000000004011dc in victim ()
#6 0x4141414141414141 in ?? ()
#7 0x4141414141414141 in ?? ()
#8 0x4141414141414141 in ?? ()
#9 0x4141414141414141 in ?? ()
#10 0x4141414141414141 in ?? ()
#11 0x4141414141414141 in ?? ()
#12 0x4141414141414141 in ?? ()
#13 0x4141414141414141 in ?? ()
#14 0x4141414141414141 in ?? ()
#15 0x4141414141414141 in ?? ()
#16 0x4141414141414141 in ?? ()
#17 0x4141414141414141 in ?? ()
#18 0x000000000004011dc in victim ()
#19 0x000000008000000018 in ?? ()
#20 0x00007ffffffffffddf0 in ?? ()
#21 0x00007ffff7dc11ca in __libc_start_call_main (main=0x4038bf0774c08548, argc=0,
argv=0xd148c6014803f8c1) at ../sysdeps/nptl/libc_start_call_main.h:58
Backtrace stopped: previous frame inner to this frame (corrupt stack?)
```

The backtrace shows that the return addresses have been overwritten by the “A” character (0x4141414141414141).

From exercise 4 we know that victim() is the function that contains the vulnerable strcpy. From the backtrace, this is confirmed with line #5 which shows the crash occurring from victim()’s execution.

The error message is SIGILL not SIGSEGV. This means that when victim() returned, it jumped to a valid address (hence no segmentation fault). But, at the valid address, the bytes there do not contain valid instructions to be ran.

SIGSEGV – when no valid address to jump to.

SIGILL – no valid instruction to execute.